

CSE 220 ■ DIGITAL SIGNAL PROCESSING

SpectraCraft

Phase Vocoder Voice Changer

Fajrin Anjum Thuja (2305034) ■ Afnan Al Habib (2305057)

Overview

Three things we can control

TIME + SPEED



Change duration
while preserving pitch

PITCH



Move the fundamental
while preserving duration

FORMANT

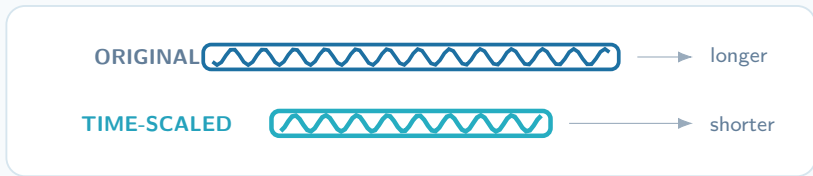


Reshape vocal-tract
resonances independently

Three independent controls. One voice.

Overview · (i) Time

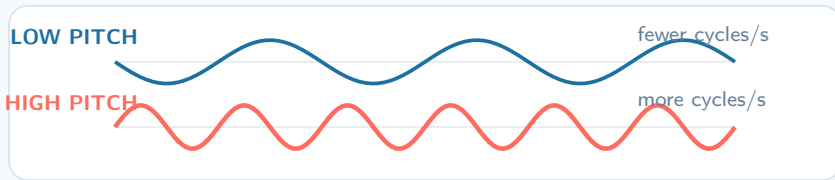
Time — playback speed



- Changing speed means changing **how long** the clip takes to play — faster or slower.
- Keeps the voice sounding the same pitch.

Overview · (ii) Pitch

Pitch — how high or low a voice sounds



- Pitch is set by the **fundamental frequency** — how fast the vocal folds vibrate.
- Fewer cycles/second = lower pitch, more cycles/second = higher pitch.
- It should change **without** touching duration.

Overview · (iii) Formant

Formant — the vocal tract's resonance shape



- Formants are the resonant “bumps” the throat and mouth add on top of the pitch.
- They give a voice its size/color and are physically separate from pitch.

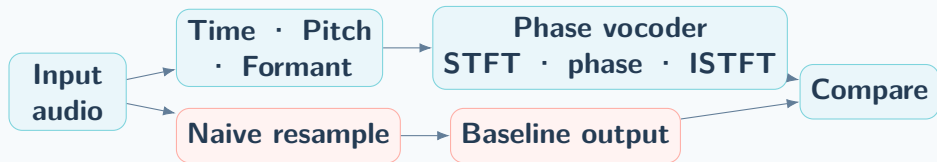
Overview · Presets

Presets = fixed combinations of the three knobs

Voice	Pitch	Speed	Formant
Male	−6 semitones	1.0×	0.80×
Female	+8 semitones	1.0×	1.30×
Child	+10 semitones	1.08×	1.45×
Robot	−12 semitones	1.0×	0.65×

Showing the project

Live demo, then under the hood



- High-level view: every clip is processed two ways and compared.
- We'll open the app now. Afterwards we come back and explain each internal step.

Pitch & time are connected

Change speed the naive way — what happens?

Speed $\times 1.5$



Duration ↓

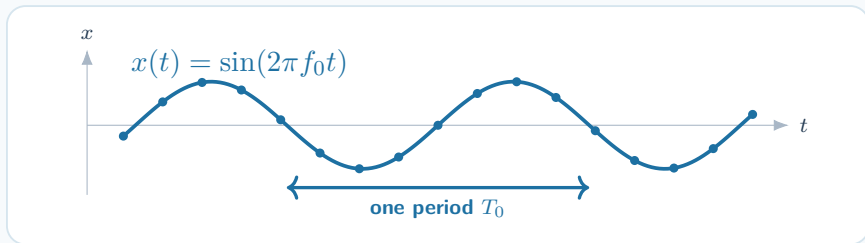


Pitch rises too

If we simply play the audio faster or slower, duration changes — but pitch moves along with it. We did not ask for that.

Why this happens — 1

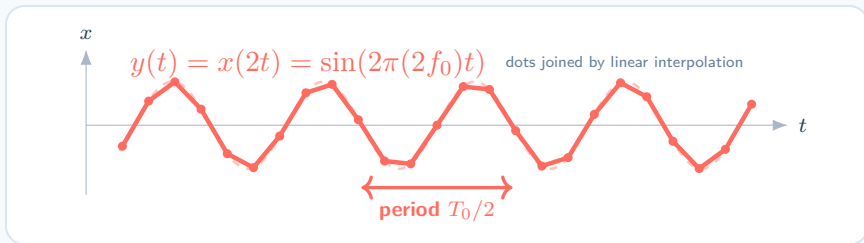
Original waveform in time



- One cycle takes T_0 seconds, so the original frequency is $f_0 = 1/T_0$.

Why this happens — 2

Interpolation evaluates the waveform at new time positions



- For $2\times$ playback, $y(t) = x(2t)$; interpolation estimates values between stored samples.
- The period becomes $T_0/2$, so $f_{\text{out}} = 2f_0$ and the pitch rises.

The core challenge

So we need to change them independently

Resampling: time + pitch coupled

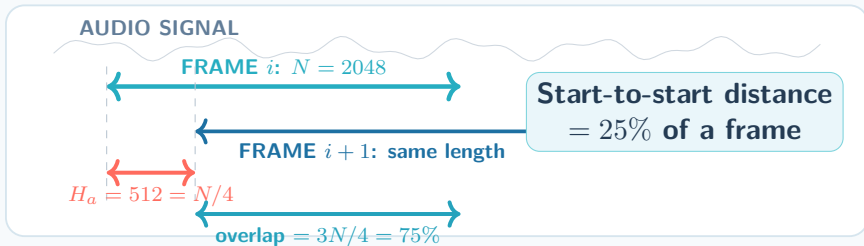


Phase vocoder: independent controls

- Simple resampling always couples time and pitch together.
- SpectraCraft's actual job is to break that coupling — using a **phase vocoder**.

Speed · Step 1

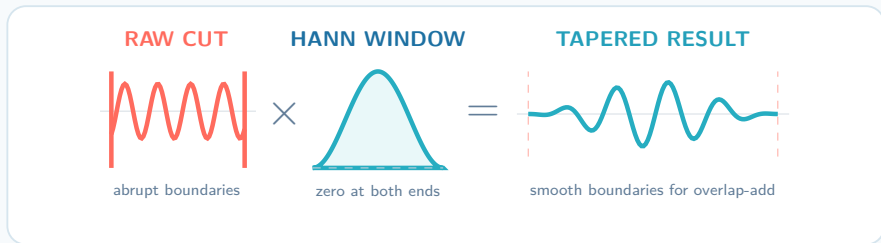
Step 1 — Time domain: frame division



- We cut the signal into overlapping frames (frame size = 2048, hop size = 512).
- Gives 75% overlap between consecutive frames.

Speed · Step 2

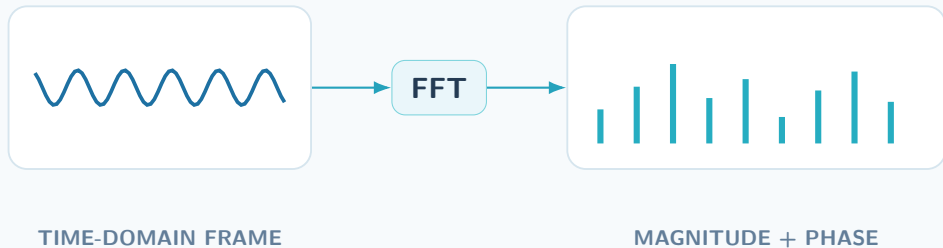
Step 2 — Apply a Hann (bell-shaped) window



- A raw rectangular frame has sharp edges — overlapping them clicks.
- Multiplying by a bell-shaped Hann window fades each frame's edges to zero.

Speed · Step 3

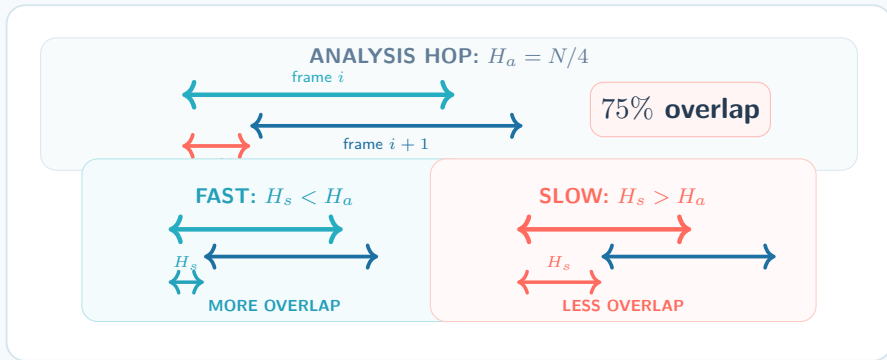
Step 3 — STFT: every frame, time $\longleftrightarrow$ frequency



- Short-Time Fourier Transform = FFT applied to each windowed frame.
- Gives magnitude and phase for every frequency bin.

Speed · Step 4

Step 4 — Why go to the frequency domain?



- We can't just add Hann-windowed time frames at new spots directly.
- We first need each frame's correct **phase**, which only exists in the frequency domain.

Speed · Step 5

Step 5 — Phase shift of every frame



Measure the phase error

$$\Delta\phi = \text{wrap}(\phi_i - \phi_{i-1} - \omega H_a)$$

Advance phase at the synthesis hop

$$\hat{\phi}_i = \hat{\phi}_{i-1} + (\omega H_a + \Delta\phi) \frac{H_s}{H_a}$$

Speed · Step 6

Step 6 — Apply the synthesis window after the IFFT



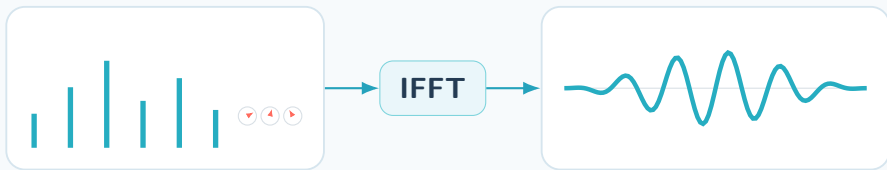
Where is the second window applied?

In the **time domain**: first take the IFFT, then multiply the reconstructed frame by a synthesis (or dual) window before overlap-add.

- With Hann analysis and synthesis windows, normalize by the accumulated window products.

Speed · Step 7

Step 7 — ISTFT: every frame, frequency $\longleftrightarrow$ time



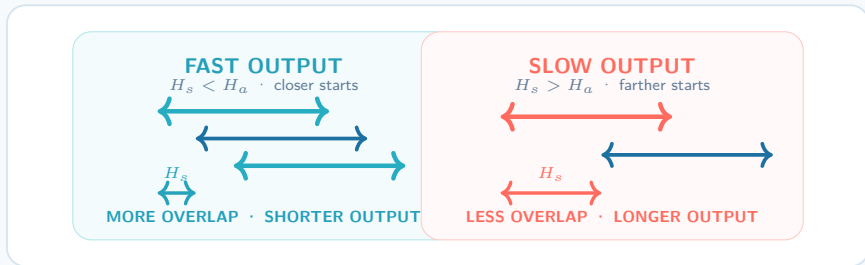
MAGNITUDE + CORRECTED PHASE

WINDOWED TIME-DOMAIN FRAME

- Each corrected frame is turned back into a short time signal with an inverse FFT.
- The synthesis window tapers the reconstructed frame before placement on the timeline.

Speed · Step 8

Step 8 — Add frames back at the new hop size



- Summed at a smaller hop (faster) or larger hop (slow), then divided by window energy.

Pitch — recap

You were right: interpolation changes pitch

Resample



Duration changes



Pitch changes

- Linear interpolation changes pitch whenever it changes duration.
- We reuse that fact on purpose for pitch shifting.

Pitch — the fix

Pitch shift = stretch duration first, then resample

$$r = 2^{\frac{\text{semitones}}{12}}$$

1. Phase-vocoder stretch

duration $\times r$, pitch unchanged



2. Resample by r

duration restored, pitch $\times r$

**The duration changes cancel;
the intended pitch shift remains.**

Pitch — what does a number mean?

What does “+8 semitones” actually mean?

Semitones	Ratio	Effect
-12	0.50×	one octave lower
-6	0.71×	noticeably deeper
+8	1.59×	frequency $\times 1.59$
+12	2.00×	one octave higher

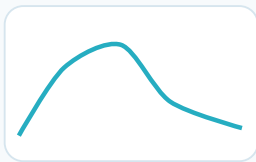
Formant · Step 1

Step 1 — Audio = pitch (source) × formant (envelope)



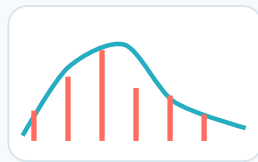
PITCH SOURCE

×



FORMANT ENVELOPE

=



VOICE SPECTRUM

- Modeled as a fast harmonic source multiplied by a slow resonant envelope.

Formant · Step 2

Step 2 — Move to log scale

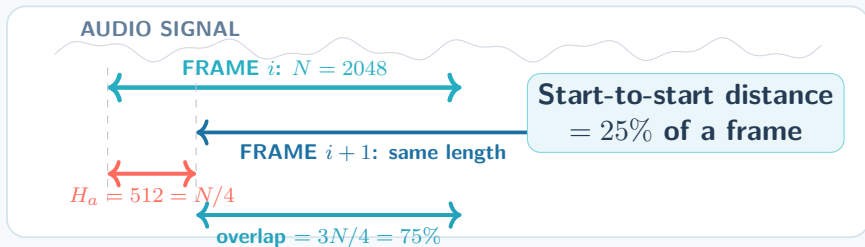


$$\log(\text{magnitude}) = \log(\text{source}) + \log(\text{envelope})$$

- Logarithms turn multiplication into addition, allowing separation.

Formant · Step 3

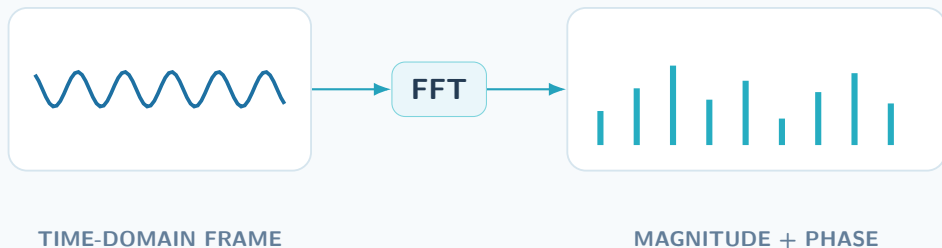
Step 3 — Frame the signal



- Divide the signal into the same overlapping frames used for speed change.

Formant · Step 4

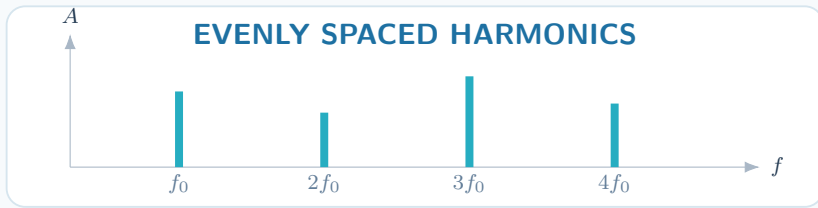
Step 4 — FFT each frame



- Each frame goes to the frequency domain to obtain magnitude.

Formant · Step 5

Step 5 — Take the log of the magnitude



- Log-magnitude keeps the source + envelope relationship additive.

Formant · Step 6

Step 6 — IFFT $\longleftrightarrow$ the cepstral domain



- Inverse FFT of log-magnitude yields the **cepstrum**.
- Slow envelope features land at low quefrency; fast pitch ripples at high quefrency.

Formant · Step 7

Step 7 — Extract the envelope

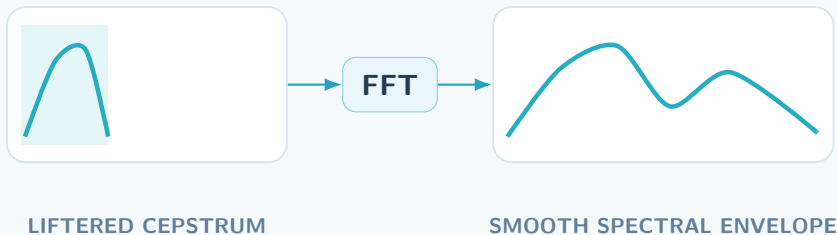


lifter = keep low quefrency, zero out the rest

liftered = cepstrum $\times$ lifter

Formant · Step 8

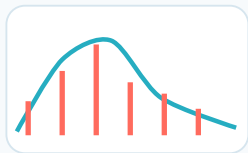
Step 8 — FFT back to the frequency domain



- Forward transform of lifted cepstrum yields the smooth spectral envelope.

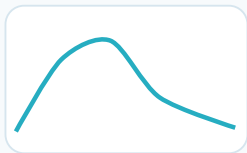
Formant · Step 9

Step 9 — Back in frequency domain: split source from filter



MAGNITUDE

÷



ENVELOPE

=



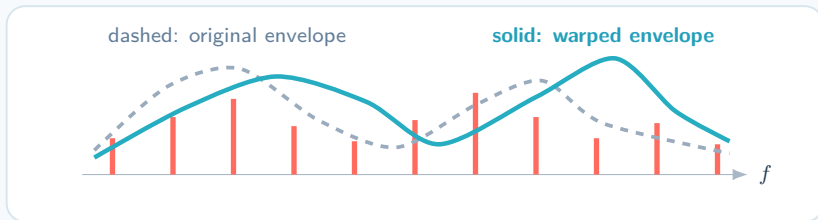
RESIDUAL HARMONICS

$$\text{residual} = \frac{\text{magnitude}}{\text{envelope}}$$

- Leaves pure pitch harmonics with the formant shape removed.

Formant · Step 10

Step 10 — Warp the envelope and multiply back



$$\text{warped} = \text{interp} \left(\frac{\text{bin}}{\text{formant_factor}}, \text{envelope} \right)$$

$$\text{new_magnitude} = \text{residual} \times \text{warped}$$

- Pushes resonances higher (brighter) or lower (deeper) without moving pitch harmonics.

Formant · Step 11

Step 11 — IFFT back to the time domain



- Combined with original phase, then ISTFT + overlap-add turns it back to audio.

Code · framing & phase update

Core code — STFT, phase correction

```
def phase_vocoder_stretch(x, stretch_factor, frame_size=2048,
    hop_analysis=512):
    hop_synthesis = round(hop_analysis * stretch_factor)
    frames = stft(x, frame_size, hop_analysis)           # framing +
        Hann + FFT
    mag, phase = abs(frames), angle(frames)
    expected = 2*pi*arange(n_bins)/frame_size * hop_analysis
    for i in range(1, n_frames):
        deviation = wrap(phase[i]-phase[i-1] - expected)
        syn_phase[i] = syn_phase[i-1] + (expected+deviation) * (
            hop_synthesis/hop_analysis)
    return istft_overlap_add(mag * exp(1j*syn_phase), frame_size,
        hop_synthesis)
```

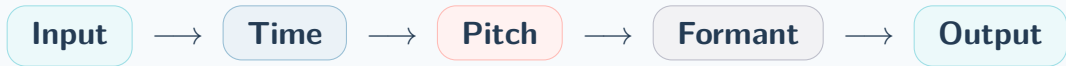
Code · pitch & formant

Core code — pitch shift, formant shift

```
def pitch_shift_phase_vocoder(x, semitones):  
    ratio = 2 ** (semitones/12)  
    stretched = phase_vocoder_stretch(x, ratio)      # duration change,  
                                                       # pitch unchanged  
    return resample_linear(stretched, ratio)         # duration undone,  
                                                       # pitch shifted  
  
def formant_shift(x, factor):  
    mag, phase = stft_magnitude_phase(x)  
    envelope = compute_spectral_envelope(mag)        # cepstral liftering  
    residual = mag / envelope  
    warped = interp(bin_idx/factor, bin_idx, envelope)  
    return istft(warped * residual * exp(1j*phase))
```

Putting it together

`process_audio()` — the full pipeline



- Runs parallel to a naive linear resample baseline for live comparisons.

SPECTRACRAFT

Thank you
Questions?

Fajrin Anjum Thuja ■ Afnan Al Habib